

CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati

saradhi@iitg.ac.in



Week #10: STL Algorithms



Algorithms - 01



- The STL provides several standard algorithms for processing elements of collections.
- These algorithms offer general fundamental services, such as searching, sorting, copying, reordering, modifying, and numeric processing.
- Algorithms are not member functions of the container classes but instead are global functions that operate with iterators.
- This has an important advantage: Instead of each algorithm being implemented for each container type, all are implemented only once for any container type.

Algorithms - 02



- The algorithm might even operate on elements of different container types.
- You can also use the algorithms for user-defined container types.
- All in all, this concept reduces the amount of code and increases the power and the flexibility of the library.

Algorithm - 03



- `#include<algorithm>`

```
vector<int> coll = { 2, 5, 4, 1, 6, 3 };  
  
auto minpos = min_element(coll.cbegin(), coll.cend());  
  
auto maxpos = max_element(coll.cbegin(), coll.cend());  
  
sort (coll.begin(), coll.end());  
  
auto pos3 = find (coll.begin(), coll.end(), 3);  
  
reverse (pos3, coll.end());
```

Algorithm - 04



- `min_element` takes range of vector elements and find the minimum element within this vector
- This returns an iterator for the position of the element found
- The next algorithm, `sort()`, as the name indicates, sorts the elements of the range defined by the two arguments.
- As usual, you could pass an optional sorting criterion.
- The default sorting criterion is operator `<`. Thus, in this example, all elements of the container are sorted in ascending order
- The last algorithm called in the example is `reverse()`, which reverses the elements of the passed range.
- Here, the third element that was found by the `find()` algorithm and the past-the-end iterator are passed as arguments



Ranges



Algorithm - 05



- A **range** is a **sequence** of elements that you can **iterate over**.
- All algorithms process one or more ranges of elements.
- An example involving ranges is given here.

Algorithm - 06



```
list<int> coll;
for (int i=20; i<=40; ++i)
{
    coll.push_back(i);
}

list<int>::iterator pos25, pos35;

pos25 = find (coll.begin(), coll.end(), 25);
pos35 = find (coll.begin(), coll.end(), 35);

cout << "max: " << *max_element (pos25, pos35) << endl;
/* exclude pos35 */

Cout << "max: " << *max_element(pos25, ++pos35) << endl;
// process last element as well.
```

Algorithm - 07



```
list<int> coll;
for (int i=20; i<=40; ++i)
{
    coll.push_back(i);
}
list<int>::iterator pos25, pos35;
pos25 = find (coll.begin(), coll.end(), 25);
pos35 = find (coll.begin(), coll.end(), 35);
pos30 = find (coll.begin(), coll.end(), 30);

if(pos30 == pos35)
{
    cout << "30 not in subrange";
}
else
{
    cout << "30 is in the subrange";
}
```

Algorithm - 08



```
list<int> coll;
for (int i=20; i<=40; ++i)
{
    coll.push_back(i);
}
list<int>::iterator pos;
pos = find_if (coll.begin(), coll.end(),
               [](int j)
               {
                   return j == 25 || j == 35;
               }
               );

if(pos30 == coll.end())
{
    cout << "No element with value 25 or 35 found";
}
```

Algorithm - 09



```
[](int j)
{
    return j == 25 || j == 35;
}
```

This is a lambda function is used as a criterion that allows a search of the first element that has either value 25 or value 35.



Multiple Ranges



Algorithm - 10



- Several algorithms process more than one range.
- In this case, you usually must define both the beginning and the end only for the first range.
- For all other ranges, you need to pass only their beginnings.
- The ends of the other ranges follow from the number of elements in the first range.
- The call of `equal()` compares all elements of the collection `coll1` element-by-element with the elements of `coll2`, beginning with its first element.

```
if( equal(coll1.begin(), coll1.end(), coll2.begin())  
{  
    cout << "All elements are equal\n";  
}
```

Algorithm - 11



```
if( equal(coll1.begin(), coll1.end(), coll2.begin())
{
    cout << "All elements are equal\n";
}
```

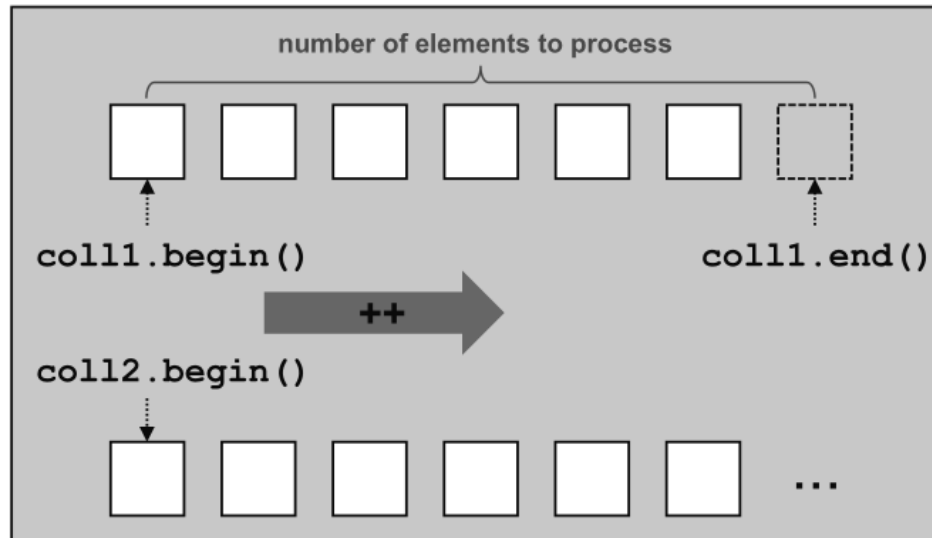
The number of elements of coll2 that are compared with the elements of coll1 is specified indirectly by the number of elements in coll1

When you call algorithms for multiple ranges, make sure that the second and additional ranges have at least as many elements as the first range

Algorithm - 12



```
if( equal(coll1.begin(), coll1.end(), coll2.begin())  
{  
    cout << "All elements are equal\n";  
}
```



Algorithm - 13



```
#include <algorithm>
#include <list>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    list<int> coll1 = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    vector<int> coll2;
    copy (coll1.cbegin(), coll1.cend(), coll2.begin());
}
```

The copy() algorithm is called above.

It copies all elements of the first range into the destination range.

The beginning and the end are defined for the first range.

Only the beginning is specified for the second range.

Algorithm - 14



```
#include <algorithm>
#include <list>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    list<int> coll1 = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    vector<int> coll2;
    copy (coll1.cbegin(), coll1.cend(), coll2.begin());
}
```

The copy algorithm overwrites rather than inserts.

So, the algorithm requires that the destination has enough elements to be overwritten.

If there is not enough room, as in this case, the result is undefined behavior.

In practice, this often means that you overwrite whatever comes after the `coll2.end()`.



Algorithm - 15



```
#include <algorithm>
#include <list>
#include <vector>
#include <deque>
using namespace std;
int main(int argc, char *argv[])
{
    list<int> coll1 = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    vector<int> coll2;
    coll2.resize (coll1.size());
    copy (coll1.cbegin(), coll1.cend(), coll2.begin());

    deque<int> coll3(coll1.size());
    copy (coll1.cbegin(), coll1.cend(), coll3.begin());
}
```

Algorithms - 16



- Different algorithms meet different needs and so can be classified by their main purposes.
- For example, some algorithms operate as read only, some modify elements, and some change the order of elements.
- This subsection gives you a brief idea of the functionality of each algorithm and in which aspect it differs from similar algorithms.
- The name of an algorithm gives you a first impression of its purpose.
- The designers of the STL introduced two special suffixes:
 - The `_if` suffix
 - The `_copy` suffix

Algorithms - 17



- **find() vs find_if find()** searches for an element that has a certain value, whereas **find_if()** searches for an element that meets the criterion passed as a function, a function object, or a lambda.
- **reverse() and reverse_copy()** - reverses the order of elements inside a range, whereas **reverse_copy()** copies the elements into another range in reverse order.



Algorithms Classification



Algorithms - 18



- **Nonmodifying algorithms**
- **Modifying algorithms**
- **Removing algorithms**
- **Mutating algorithms**
- **Sorting algorithms**
- **Sorted-range algorithms**
- **Numeric algorithms.**



Non-modifying Algorithms



Algorithms - 19



- Non-modifying algorithms change neither the order nor the value of the elements they process
- These algorithms operate with input and forward iterators
- These can be called for all standard containers.

Algorithms - 20



Name	Effect
<code>for_each()</code>	Performs an operation for each element
<code>count()</code>	Returns the number of elements
<code>count_if()</code>	Returns the number of elements that match a criterion
<code>min_element()</code>	Returns the element with the smallest value
<code>max_element()</code>	Returns the element with the largest value
<code>minmax_element()</code>	Returns the elements with the smallest and largest value (since C++11)
<code>find()</code>	Searches for the first element with the passed value
<code>find_if()</code>	Searches for the first element that matches a criterion
<code>find_if_not()</code>	Searches for the first element that matches a criterion not (since C++11)
<code>search_n()</code>	Searches for the first n consecutive elements with certain properties
<code>search()</code>	Searches for the first occurrence of a subrange
<code>find_end()</code>	Searches for the last occurrence of a subrange
<code>find_first_of()</code>	Searches the first of several possible elements
<code>adjacent_find()</code>	Searches for two adjacent elements that are equal (by some criterion)

There are many more useful functions in this classification

The syntax of `count_if` is as follows

```
count_if(begin_iterator, end_iterator, predicate);
```



Example count_if



Algorithms - 21



```
#include <algorithm>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int ec = count_if( n.begin(), n.end(),
                      [](int x)
                      {
                          return x % 2 == 0;
                      }
                      );
    cout << "Even numbers: " << ec << endl;
}
```

Algorithms - 22



```
#include <algorithm>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    vector<string> w = {"apple", "bat", "cat", "elephant", "dog"};
    int lw = count_if( w.begin(), w.end(),
                      [](int x)
                      {
                          return w.length() > 3;
                      }
                    );
    cout << "Words with length > 3: " << lw << endl;
}
```



Modifying Algorithms



Algorithms - 23



- **Modifying algorithms change the value of elements.**
- **Such algorithms might modify the elements of a range directly**
- **Modify them while they are being copied into another range.**
- **If elements are copied into a destination range, the source range is not changed.**

Algorithms - 24



Name	Effect
<code>for_each()</code>	Performs an operation for each element
<code>copy()</code>	Copies a range starting with the first element
<code>copy_if()</code>	Copies elements that match a criterion (since C++11)
<code>copy_n()</code>	Copies n elements (since C++11)
<code>copy_backward()</code>	Copies a range starting with the last element
<code>move()</code>	Moves elements of a range starting with the first element (since C++11)
<code>move_backward()</code>	Moves elements of a range starting with the last element (since C++11)
<code>transform()</code>	Modifies (and copies) elements; combines elements of two ranges
<code>merge()</code>	Merges two ranges
<code>swap_ranges()</code>	Swaps elements of two ranges
<code>fill()</code>	Replaces each element with a given value
<code>fill_n()</code>	Replaces n elements with a given value
<code>generate()</code>	Replaces each element with the result of an operation
<code>generate_n()</code>	Replaces n elements with the result of an operation
<code>iota()</code>	Replaces each element with a sequence of incremented



Example for_each



Algorithms - 24



```
#include <algorithm>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int ec = for_each( n.begin(), n.end(),
                      [](int x)
                      {
                          cout << x << " ";
                      }
                    );

    cout << endl;
}
```

for_each algorithm is used to **apply a given function to every element in a range.**

Algorithms - 25



```
#include <algorithm>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int ec = for_each( n.begin(), n.end(),
                      [](int& x)
                      {
                          x = x * 2;
                      }
                    );
    for (int n : numbers)
    {
        cout << n << " ";
    }
    cout << endl;
}
```

Algorithms - 26



```
#include <algorithm>
#include <vector>
using namespace std;
void print_element(int x)
{
    cout << x << " " ;
}

int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int ec = for_each( n.begin(), n.end(), print_element);
    cout << endl;
}
```

Algorithms - 27



```
#include <algorithm>
#include <vector>
using namespace std;
class pe
{
    void operator(int x) const
    {
        cout << x << " " ;
    }
};
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    int ec = for_each( n.begin(), n.end(), pe());
    cout << endl;
}
```



Example copy



Algorithms - 28



```
#include <algorithm>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    vector<int> dest(n.size());

    copy( n.begin(), n.end(), dest.begin());
    cout << endl;
}
```

copy function copies all elements from a source range to a destination range.

Syntax: copy(begin_source, end_source, begin_destination);



Example copy_if



Algorithms - 29



```
#include <algorithm>
#include <vector>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    vector<int> dest;

    dest.reserve(n.size());

    copy_if( n.begin(), n.end(), back_inserter(dest),
            [] (int x)
            {
                return x % 2 == 0;
            }
            );
    cout << endl;
}
```

Algorithms - 30



- **back_inserter** is an **iterator adapter** provided by the C++ Standard Library in the `<iterator>` header.
- It is used to **insert elements at the end** of a container, typically in conjunction with STL algorithms like `std::copy`, `std::copy_if`, and others.
- It creates an **output iterator** that allows elements to be added to the end of a **sequence container** (such as `std::vector`, `std::deque`, or `std::list`) using the container's **push_back()** method.
- It is particularly useful when the destination container may not have a predefined size or when you want to dynamically add elements as they are copied or generated by an algorithm.
- The example presented above is an usecase of `back_inserter`



Removing Algorithms



Algorithms - 31



- Removing algorithms are a special form of modifying algorithms.
- They can remove the elements either in a single range or while these elements are being copied into another range.
- As with modifying algorithms, you can't use an associative or unordered container as a destination, because the elements of these containers are considered to be constant..

Name	Effect
<code>remove()</code>	Removes elements with a given value
<code>remove_if()</code>	Removes elements that match a given criterion
<code>remove_copy()</code>	Copies elements that do not match a given value
<code>remove_copy_if()</code>	Copies elements that do not match a given criterion
<code>unique()</code>	Removes adjacent duplicates (elements that are equal to their predecessor)
<code>unique_copy()</code>	Copies elements while removing adjacent duplicates

Algorithms - 32



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    string t = "Hello World You are so round";

    auto new_end = remove(t.begin(), t.end(), 'o');
    t.erase(new_end, t.end());
    cout << endl;
}
```

Algorithms - 33



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 2, 5, 2, 6};
    auto new_end = remove(n.begin(), n.end(), 2);

    n.erase(new_end, n.end());
}
```



Example: remove_if



Algorithms - 34



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 2, 5, 2, 6};
    auto new_end = remove_if(n.begin(), n.end(),
        [] (int x)
        {
            return x %2 == 0;
        }
    );

    n.erase(new_end, n.end());
}
```




Mutating Algorithms



Algorithms - 35



- Mutating algorithms are algorithms that change the order of elements (and not their values) by assigning and swapping their values.

Name	Effect
<code>reverse()</code>	Reverses the order of the elements
<code>reverse_copy()</code>	Copies the elements while reversing their order
<code>rotate()</code>	Rotates the order of the elements
<code>rotate_copy()</code>	Copies the elements while rotating their order
<code>next_permutation()</code>	Permutates the order of the elements
<code>prev_permutation()</code>	Permutates the order of the elements
<code>shuffle()</code>	Brings the elements into a random order (since C++11)
<code>random_shuffle()</code>	Brings the elements into a random order
<code>partition()</code>	Changes the order of the elements so that elements that match a criterion are at the front
<code>stable_partition()</code>	Same as <code>partition()</code> but preserves the relative order of matching and nonmatching elements
<code>partition_copy()</code>	Copies the elements while changing the order so that elements that match a criterion are at the front



Example: next_permutation



Algorithms - 36



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3};
    do
    {
        for (int j : n )
        {
            cout << j << " ";
        }
        cout << endl;
    }while( next_permutation(n.begin(), n.end() ) );
}
```

Rearranges the elements in a container to generate the **next lexicographically greater permutation**



Syntax: **next_permutation(begin, end);**

Algorithms - 37



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    shuffle(n.begin(), n.end(), default_random_engine(10));
    for (int j : n )
    {
        cout << j << " ";
    }
    cout << endl;
}
```

Randomly shuffles the elements in a container. It uses a **random number generator (RNG)** to produce different orders of the elements



Syntax: `shuffle(begin, end, rng);`

Algorithms - 38



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> n = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    partition(n.begin(), n.end(),
              [] (int x)
              {
                  return x > 5;
              }
              );
}
```

All numbers greater than 5 are moved to the front of the vector, and numbers less than or equal to 5 are moved to the back.



Sorting Algorithms



Algorithms - 39



- Sorting algorithms are a special kind of mutating algorithm because they also change the order of the elements.

Name	Effect
<code>sort()</code>	Sorts all elements
<code>stable_sort()</code>	Sorts while preserving order of equal elements
<code>partial_sort()</code>	Sorts until the first n elements are correct
<code>partial_sort_copy()</code>	Copies elements in sorted order
<code>nth_element()</code>	Sorts according to the n th position
<code>partition()</code>	Changes the order of the elements so that elements that match a criterion are at the beginning
<code>stable_partition()</code>	Same as <code>partition()</code> but preserves the relative order of matching and nonmatching elements
<code>partition_copy()</code>	Copies the elements while changing the order so that elements that match a criterion are at the beginning
<code>make_heap()</code>	Converts a range into a heap
<code>push_heap()</code>	Adds an element to a heap
<code>pop_heap()</code>	Removes an element from a heap
<code>sort_heap()</code>	Sorts the heap (it is no longer a heap after the call)

Algorithms - 40



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> numbers = {3, 1, 4, 1, 5, 9, 2, 6, 5};
    sort(n.begin(), n.end(),
        [] (int a, int b)
        {
            return a > b;
        }
    );
}
```

is a **non-stable** sorting algorithm that rearranges elements in a container in ascending order by default,

Syntax: `sort(begin, end);`

Algorithms - 41



```
#include <algorithm>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    vector<int> numbers = {3, 1, 4, 1, 5, 9, 2, 6, 5};
    sort(n.begin(), n.end(),
        [] (int a, int b)
        {
            return a > b;
        }
    );
}
```

stable sort preserves the relative order of equal elements.

Syntax: `stable_sort(begin, end, comp);`



Thank You





Questions?

